



CENG 499

Introduction to Machine Learning

Spring 2024 - 2025

Programming Assignment 2 - SVMs with Kernels

Due date: 27 04 2025, Friday, 23:59

1 Overview

In this assignment, you will implement Support Vector Machine (SVM) classifiers from scratch, incorporating three distinct kernel functions: Polynomial, Radial Basis Function (RBF), and a custom kernel tailored for time series data. These implementations will be evaluated on two temporal datasets—weather and air quality—using a time series-specific k-fold cross-validation approach to optimize hyperparameters. Furthermore, you will address class imbalance issues to enhance model robustness. The primary goal of this assignment is to deepen your understanding of SVMs, explore the role of kernel methods in handling non-linear relationships, and tackle practical challenges associated with temporal data analysis.

Keywords: *SVM, kernels, time series, k-fold cross-validation, data imbalance, weather dataset, air quality dataset*

2 Introduction

Support Vector Machines (SVMs) are a cornerstone of supervised learning, renowned for their ability to perform classification by identifying the optimal hyperplane that maximizes the margin between classes in a high-dimensional space. When data is not linearly separable in its original feature space, SVMs leverage kernel functions to transform the data into a higher-dimensional space where a linear boundary can be established. This assignment requires you to build an SVM classifier from scratch, including the optimization of its dual form and the integration of kernel functions, without using pre-existing SVM implementations such as `scikit-learn`'s `SVC`. However, you are permitted to use `numpy` for numerical operations and `pandas` for data manipulation.

You will apply your SVM implementation to two temporal datasets:

- **Weather Dataset:** This dataset records daily minimum temperatures in Melbourne from 1981 to 1990, offering a time series with potential seasonal patterns and trends.
- **Air Quality Dataset:** This dataset tracks the Air Quality Index (AQI) over time, reflecting pollution levels with inherent temporal dependencies.

Your tasks encompass exploratory data analysis (EDA) to uncover dataset characteristics, implementing SVMs with various kernels, optimizing hyperparameters using a time series-adapted k-fold cross-validation, and mitigating class imbalance to ensure fair and effective classification. This assignment will equip you with practical skills in SVM development and insights into handling sequential data.

3 Tasks

3.1 Task 1: Dataset Acquisition and Preparation

Your first task is to acquire and preprocess two temporal datasets for classification. These datasets will serve as the foundation for your Support Vector Machine (SVM) implementation. This section outlines the steps to source, load, preprocess, and transform the data into a format suitable for binary classification.

- **Dataset Sources:**

- **Weather Dataset:** Obtain the "Daily Minimum Temperatures in Melbourne" dataset, which contains daily minimum temperature readings over a decade (1981–1990). This dataset is widely used for time series analysis and is available for download from this GitHub repository. Alternatively, you may find preprocessed versions on Kaggle by searching for "Daily Minimum Temperatures in Melbourne." The dataset includes two columns: 'Date' and 'Temp' (temperature in Celsius).
- **Air Quality Dataset:** Acquire the "Air Quality Data in India (2017 - 2022)" dataset, which provides hourly PM2.5 measurements, a key indicator of air pollution. This dataset is accessible on Kaggle. It contains the following columns: "Timestamp", "Year", "Month", "Day", "Hour", and "PM2.5". Note that you will need to preprocess this hourly data into daily averages for consistency with the classification task.

- **Data Loading:** Load each dataset into a `pandas` DataFrame to enable efficient manipulation and analysis. Use the `pandas.read_csv()` function, ensuring that the temporal structure (e.g., date or timestamp columns) is preserved during loading. For the weather dataset, the 'Date' column should be parsed directly as a date type (e.g., using the `parse_dates` parameter). For the air quality dataset, the 'Timestamp' column or the combination of 'Year', 'Month', 'Day', and 'Hour' columns will serve as the temporal reference.

- **Data Preprocessing:**

- **Weather Dataset:** Since this dataset is already structured with daily temperature readings, no additional preprocessing is required beyond loading it into a DataFrame. Verify that the 'Date' and 'Temp' columns are correctly formatted and free of missing values before proceeding.
- **Air Quality Dataset:** The hourly PM2.5 data must be aggregated into daily averages to align with the daily granularity of the weather dataset and the classification task. Follow these steps:
 1. Create a 'Date' column by combining the 'Year', 'Month', and 'Day' columns into a single date field. Use `pd.to_datetime` with a dictionary of the form `{ 'year': 'Year', 'month': 'Month', 'day': 'Day' }` to construct the dates efficiently.
 2. Group the data by the 'Date' column and calculate the mean PM2.5 value for each day using `groupby('Date').mean()`. This reduces the dataset from hourly to daily resolution.
 3. Drop all columns except 'Date' and 'PM2.5', resulting in a simplified dataset with only these two fields: 'Date' (e.g., "2020-01-01") and 'PM2.5' (daily average value).
 4. Validate the resulting DataFrame to ensure no duplicate dates or inconsistent values remain.

- **Classification Problem Formulation:**

- **Weather Dataset:** Transform the dataset into a binary classification problem by predicting whether the next day’s temperature exceeds the dataset’s mean temperature. Calculate the mean temperature as:

$$\mu = \frac{1}{N} \sum_{i=1}^N T_i$$

where T_i is the temperature on day i , and N is the total number of days. For each day i , compute the label for day $i + 1$: assign a positive label (1) if $T_{i+1} > \mu$, and a negative label (0) if $T_{i+1} \leq \mu$. This creates a balanced classification task based on historical trends.

- **Air Quality Dataset:** Convert the daily average PM2.5 data into a binary classification task by predicting whether the next day’s PM2.5 level indicates ”unhealthy” conditions. Based on standard air quality thresholds, use $PM2.5 > 100$ as the criterion for unhealthy air. For each day i , assign a positive label (1) if the PM2.5 value for day $i + 1$ exceeds 100, and a negative label (0) if it is ≤ 100 . This reflects real-world air quality monitoring practices.
- **Feature Engineering:** Use a sliding window approach to create input features from past observations, enabling the SVM to learn temporal patterns:
 - **Weather Dataset:** Use the temperatures from the previous 7 days ($T_{i-6}, T_{i-5}, \dots, T_i$) as features to predict the label for day $i + 1$. For example, for a target day of January 8th, the feature vector would include temperatures from January 1st to January 7th. This 7-day window captures weekly trends in temperature data.
 - **Air Quality Dataset:** Similarly, use the daily average PM2.5 values from the previous 7 days ($PM2.5_{i-6}, PM2.5_{i-5}, \dots, PM2.5_i$) as features to predict the label for day $i + 1$. After preprocessing, the hourly data is already aggregated into daily values, so the 7-day window applies directly to the ’PM2.5’ column. This ensures consistency with the weather dataset’s feature structure.
- **Preprocessing:** To prepare the feature vectors for SVM training, perform the following steps:
 - **Handling Missing Values:** Check both datasets for missing entries. Address gaps by either interpolating missing values (e.g., using linear interpolation with `pandas.interpolate()`) or dropping affected rows if the gaps are minimal. For the air quality dataset, ensure that daily averages are computed only from complete days (i.e., exclude days with excessive missing hourly data).
 - **Normalization:** Normalize all features to a consistent scale to prevent biases in SVM optimization due to differing magnitudes. Options include scaling to the range $[0, 1]$ using min-max normalization or applying standard normalization (zero mean, unit variance) with `sklearn.preprocessing.StandardScaler`. Apply the same normalization method to both datasets for consistency.
 - **Data Validation:** After preprocessing, verify that the feature vectors and labels align correctly (e.g., each 7-day window corresponds to the correct next-day label) and that the datasets are free of anomalies such as negative PM2.5 values or unrealistic temperatures.

This preparation ensures that the datasets are structured appropriately for time series classification with SVMs.

3.2 Task 2: Exploratory Data Analysis

Conduct a thorough exploratory data analysis (EDA) to understand the properties and temporal dynamics of both datasets before proceeding to model implementation.

- **Summary Statistics:** For each dataset, compute and display summary statistics using `df.describe()` in `pandas`. This includes the mean, standard deviation, minimum, maximum, and quartiles for each feature:
 - For the weather dataset, analyze the past 7 days' temperatures (e.g., `Temp_1` to `Temp_7`).
 - For the air quality dataset, analyze the past 7 days' daily average PM2.5 values (e.g., `PM2.5_1` to `PM2.5_7`).

These statistics provide insights into the data's central tendency, variability, and range, which are critical for interpreting SVM performance and kernel effectiveness.

- **Time Series Visualization:** Plot the time series data to observe temporal patterns:
 - For the weather dataset, create a line plot of daily minimum temperature versus date using `matplotlib` or `seaborn`, highlighting trends or seasonal cycles (e.g., yearly fluctuations).
 - For the air quality dataset, plot the daily average PM2.5 versus date, noting any spikes, periodicities (e.g., seasonal variations), or anomalies that might influence classification.

Include these plots in your report with captions explaining key observations, such as long-term trends or recurring patterns.

- **Class Distribution Analysis:** Examine the distribution of binary labels (positive vs. negative) for each dataset:
 - Use a histogram or pie chart to visualize the proportion of each class:
 - * For the weather dataset, visualize the distribution of days where the next day's temperature exceeds the mean.
 - * For the air quality dataset, visualize the distribution of days where the next day's PM2.5 level exceeds 100.
 - Calculate the imbalance ratio (e.g., ratio of minority to majority class) to quantify any skew, which will inform your approach to handling imbalance in Task 6.

Incorporate these analyses into your report, providing detailed descriptions alongside figures to justify your subsequent modeling decisions.

3.3 Task 3: Implement SVM

Develop a Support Vector Machine classifier from scratch, focusing on the dual form optimization and prediction logic. You must not use pre-built SVM implementations (e.g., `scikit-learn`'s `SVC`), though `numpy` and `scipy` are allowed for numerical computations and optimization support.

Your implementation should include:

- **Class Structure:** Define a class `SVM` with:
 - `__init__(C=1.0, kernel=None)`: Initialize the SVM with a regularization parameter C and a kernel function (to be implemented in Task 4).
 - `fit(X, y)`: Train the SVM by solving the dual optimization problem to determine the Lagrange multipliers and bias term.
 - `predict(X)`: Classify new samples based on the trained model.
- **Dual Form Optimization:** Formulate and solve the dual problem:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

subject to the constraints:

$$0 \leq \alpha_i \leq C \quad \text{for all } i = 1, \dots, n$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

Here, α_i are the Lagrange multipliers, $y_i \in \{-1, 1\}$ are the binary labels, $K(\mathbf{x}_i, \mathbf{x}_j)$ is the kernel function, and C controls the trade-off between margin maximization and classification error. This optimization identifies the support vectors (samples where $\alpha_i > 0$).

- **Optimization Approach:** Implement a method to solve this quadratic programming problem:
 - You may adapt a simplified Sequential Minimal Optimization (SMO) algorithm, which iteratively updates pairs of α_i values while satisfying the constraints.
 - Alternatively, use `scipy.optimize.minimize` to solve the dual problem, ensuring that your code constructs the objective function and constraints manually.
 - In a more efficient case you can use a Quadratic Programming solver `cvxopt.solvers.qp` after creating the necessary matrices.
- **Bias Calculation:** Compute the bias term b using the support vectors:

$$b = \frac{1}{N_s} \sum_{i \in S} \left(y_i - \sum_{j \in S} \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i) \right)$$

where S is the set of support vector indices, and N_s is the number of support vectors.

- **Prediction Function:** Implement prediction as:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right)$$

This determines the class label based on the sign of the decision function.

This task builds the core SVM machinery, preparing you to integrate kernel functions in the next step.

3.4 Task 4: Implement Kernel Functions

Implement four kernel functions to enable the SVM to capture different types of relationships in the data. Each kernel must be coded from scratch without relying on library implementations.

- **Linear Kernel:**

$$K(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$$

This kernel computes the dot product between two feature vectors, assuming linear separability in the original space. It serves as a baseline for comparison with more complex kernels.

- **Polynomial Kernel:**

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + c)^d$$

Here, c (e.g., 1) is a constant that shifts the kernel, and d is the polynomial degree (e.g., 2 or 3). This kernel introduces non-linearity by considering higher-order interactions between features, enabling the SVM to model polynomial decision boundaries.

- **Radial Basis Function (RBF) Kernel:**

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}\right)$$

The RBF kernel uses a bandwidth parameter σ to measure similarity based on Euclidean distance. It maps data into an infinite-dimensional space, making it highly flexible for capturing complex, non-linear patterns. The choice of σ significantly affects the kernel's behavior, requiring careful tuning.

- **Custom Time Series Kernel:** It is explained in the following subsection.

3.5 Custom Time Series Kernel

The custom kernel is specifically tailored for time series data, where temporal ordering, trends, and seasonality play critical roles. It integrates three key components—linear, periodic, and RBF—while incorporating lag weighting to account for dependencies across time steps.

3.5.1 Motivation

Time series data often exhibits:

- **Temporal Ordering:** The sequence of observations is crucial; past values influence future ones.
- **Autocorrelation:** Values at different time lags are correlated.
- **Trends and Seasonality:** Long-term movements or periodic patterns may be present.

To capture these characteristics, the custom kernel combines:

- A **linear component** to model trends or overall similarity.
- A **periodic component** to capture repeating patterns (e.g., daily or weekly cycles).
- An **RBF component** to account for local similarity between time steps.
- **Lag weighting** to emphasize similarities between nearby time steps while considering temporal shifts.

This design draws inspiration from Gaussian Processes (GPs), which use composite kernels to model time series, and Recurrent Neural Networks (RNNs), which handle sequential dependencies through hidden states.

3.5.2 Mathematical Formulation

For two time series vectors $\mathbf{x} = [x_1, x_2, \dots, x_T]$ and $\mathbf{y} = [y_1, y_2, \dots, y_T]$, the custom kernel is defined as:

$$K(\mathbf{x}, \mathbf{y}) = \sum_{\tau=-L}^L w_{\tau} \cdot \sum_{t=1}^T [w_{\text{lin}} \cdot K_{\text{lin}}(x_t, y_{t+\tau}) + w_{\text{per}} \cdot K_{\text{per}}(x_t, y_{t+\tau}) + w_{\text{rbf}} \cdot K_{\text{rbf}}(x_t, y_{t+\tau})]$$

where:

- τ is the lag, ranging from $-L$ to L , allowing the kernel to consider temporal shifts.
- $w_{\tau} = \exp\left(-\frac{\tau^2}{2\sigma_{\tau}^2}\right)$: Gaussian weights that decrease as the lag increases, controlled by σ_{τ} .
- $K_{\text{lin}}(x_t, y_{t+\tau}) = x_t \cdot y_{t+\tau}$: Linear kernel for trend similarity.
- $K_{\text{per}}(x_t, y_{t+\tau}) = \exp\left(-\frac{2 \sin^2\left(\frac{\pi(x_t - y_{t+\tau})}{p}\right)}{\sigma^2}\right)$: Periodic kernel with period p (e.g., 7 for weekly cycles).
- $K_{\text{rbf}}(x_t, y_{t+\tau}) = \exp\left(-\frac{(x_t - y_{t+\tau})^2}{2\sigma^2}\right)$: RBF kernel for local similarity.
- $w_{\text{lin}}, w_{\text{per}}, w_{\text{rbf}}$: Weights summing to 1, balancing the contributions of each component.

This formulation ensures that the kernel captures both aligned and shifted similarities, weighted by their temporal proximity.

3.5.3 Implementation Details

- **Lag Handling:** For each lag τ , the kernel computes similarities between x_t and $y_{t+\tau}$, adjusting indices to avoid out-of-bounds errors.
- **Efficiency:** The double summation over τ and t is computationally intensive for large T or L . For this assignment, $L = 3$ and $T = 7$ (weather) or 24 (AQI), making it feasible.
- **Hyperparameters:** L , σ_τ , p , σ , and the weights w_{lin} , w_{per} , w_{rbf} can be tuned via cross-validation to optimize performance.

This kernel's design allows the SVM to model complex temporal patterns, making it particularly effective for time series classification tasks. Test each kernel function independently to ensure correctness before integrating them into the SVM.

3.6 Task 5: Train and Evaluate SVM Models

Train and assess your SVM implementation with the implemented kernels on both datasets:

- **Dataset Split:** Perform a temporal split for each dataset: use the first 80% of the data (chronologically) for training and the last 20% for testing. This preserves the temporal order, mimicking real-world forecasting scenarios.
- **Hyperparameter Tuning:** Optimize hyperparameters using time series k-fold cross-validation ($k = 5$):
 - Divide the training set into 5 sequential folds, ensuring that each fold’s test data follows its training data in time.
 - Tune C (e.g., $[0.1, 1, 10]$), d for polynomial (e.g., $[2, 3]$), σ for RBF and custom kernels (e.g., $[0.1, 1, 10]$), and custom kernel weights (e.g., $w_{\text{lin}} = w_{\text{per}} = w_{\text{rbf}} = 1/3$ vs. other combinations).
 - Select the hyperparameter set with the highest average cross-validation accuracy.
- **Evaluation Metrics:** On the test set, compute:
 - **Accuracy:** $\frac{\text{Number of correct predictions}}{\text{Total test samples}}$.
 - **Precision:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$.
 - **Recall:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$.
 - **F1-Score:** $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.

Report these metrics for each kernel on both datasets, discussing in your report which kernel performs best and why, considering the temporal nature of the data.

3.7 Bonus Task: Handle Data Imbalance

Investigate and mitigate class imbalance to improve model performance:

- **Imbalance Assessment:** Using the class distribution from Task 2, determine if either dataset exhibits significant imbalance (e.g., minority class $< 30\%$ of total). Report the imbalance ratio for each dataset.
- **Imbalance Handling:** Implement at least one technique:
 - **Class Weighting:** Modify the SVM objective by assigning higher weights to the minority class in the regularization term, adjusting C as $C_{\text{minority}} = C \cdot \frac{N_{\text{majority}}}{N_{\text{minority}}}$.
 - **Oversampling:** Manually implement a basic oversampling method (e.g., duplicate minority class samples) or use SMOTE (Synthetic Minority Oversampling Technique) via `imblearn`, ensuring temporal order is respected.
- **Re-evaluation:** Retrain the SVM with the best-performing kernel (from Task 5) on the balanced training set, re-evaluate on the test set, and compare performance metrics (especially recall and F1-score) before and after imbalance correction. Discuss the impact in your report.

4 Submission

Submit via ODTUClass a zip file named `pa2.zip` containing:

- **Python Code:** Your complete implementation, including data preparation, EDA, SVM, kernel functions, and imbalance handling.
- **Report:** A PDF document detailing:
 - EDA results: Summary statistics, time series plots, and class distributions.
 - Hyperparameter tuning outcomes and final selected parameters.
 - Performance metrics for all kernels before and after imbalance handling.
 - Discussion of kernel effectiveness and the impact of imbalance mitigation.

Note: Do NOT include virtual environments in your submission.

5 Regulations

- **Programming Language:** Your program must be coded in Python 3. It will be tested in a virtual environment with standard libraries like (`numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn`, `cvxopt`). Additional libraries can be installed if needed for your implementation.
- **Late Submission:** Late submissions are allowed with a penalty of $10 \times$ day. You are only allowed two days of late submission.
- **Cheating:** All submitted work must be your own. Using third-party code or large-language models (LLMs) directly (copy-paste) is considered cheating and will result in disciplinary action under our "zero tolerance" policy.
- **Newsgroup:** Follow ODTUClass for updates and discussions daily.
- **Grading:** This homework is worth 100 points and constitutes 6% of your total grade.